

C Roadmap Version 2

This version fixes the earlier issue by providing actual coding questions, not placeholders. Every concept below has 15 beginner coding tasks, 20 intermediate coding tasks, and 30 advanced coding tasks.

Roadmap order

1. Setup, compilation, and execution model
2. Variables, types, operators, and conversions
3. Input, output, and formatted text processing
4. Control flow, functions, and recursion
5. Arrays, strings, and character handling
6. Pointers, const, and pointer arithmetic
7. Structs, unions, enums, and typedefs
8. Dynamic memory, lifetime, and ownership
9. Preprocessor, headers, modular design, and builds
10. File handling, binary data, and parsing
11. Bit manipulation, endianness, and low-level representation
12. Function pointers, callbacks, and generic programming
13. Data structures in C
14. Algorithms, complexity, and problem solving in C
15. Debugging, testing, sanitizers, and tooling
16. System programming, concurrency, embedded C, and expert-level pitfalls

How to use this

- Solve beginner tasks first without looking up solutions immediately.
- Push each finished task to GitHub with a short README.
- For every intermediate task, write tests and note failure cases.
- For every advanced task, add tooling: warnings, sanitizers, and documentation.
- Do not mark a concept complete until you can explain, code, debug, and improve it.

1. Setup, compilation, and execution model

Beginner coding questions

1. Write the smallest valid C program that prints your name.
2. Write a program that returns exit code 0 on success and 1 on failure.
3. Compile a two-line program manually using gcc and explain each stage.
4. Write a program with one syntax error and identify why the compiler rejects it.
5. Write a program with one warning and fix it until the warning disappears.
6. Create a file with main() and print the number of command-line arguments.
7. Write a program that prints every argv entry on a new line.
8. Create a program that uses a helper function declared before main().
9. Split one program into two .c files and one .h file.
10. Write a program that uses #include with your own header.
11. Write a program that demonstrates the difference between compile error and runtime error.
12. Create a Makefile target that builds a simple hello program.
13. Write a program and intentionally omit a semicolon, then explain the error.
14. Write a program that prints FILE and LINE.
15. Build the same program with -Wall -Wextra and list all warnings found.

Intermediate coding questions

1. Build a mini project around setup, compilation, and execution model that takes input, validates errors, and has at least three functions.
2. Write a broken program involving setup, compilation, and execution model and then fix the bug with a short explanation.
3. Implement one solution for setup, compilation, and execution model using arrays and another using pointers, then compare them.
4. Write test cases for a setup, compilation, and execution model function covering normal, edge, and invalid input.
5. Refactor a messy setup, compilation, and execution model program into modular .c and .h files.
6. Write code for setup, compilation, and execution model that fails safely on invalid input instead of crashing.
7. Implement a command-line tool that demonstrates setup, compilation, and execution model in a practical scenario.
8. Write a version of a setup, compilation, and execution model solution with one hidden bug and challenge yourself to detect it using gdb or prints.
9. Measure the time or operation count of two setup, compilation, and execution model approaches and explain the trade-off.
10. Write a setup, compilation, and execution model program that avoids magic numbers by using named constants, enums, or macros.
11. Design an API for setup, compilation, and execution model with clear ownership and error-return rules.

12. Write a program where setup, compilation, and execution model interacts with file input or command-line input.
13. Implement boundary checks for every buffer, index, or allocation used in a setup, compilation, and execution model exercise.
14. Create a GitHub-ready README for one setup, compilation, and execution model exercise explaining approach, complexity, and failure cases.
15. Write a setup, compilation, and execution model solution and then rewrite it to be more portable across compilers or platforms.
16. Introduce logging into a setup, compilation, and execution model mini project and use it to trace execution.
17. Write a version of a setup, compilation, and execution model exercise that handles memory cleanup through one cleanup label.
18. Create a set of interview-style oral questions from one setup, compilation, and execution model coding problem and answer them yourself.
19. Write a stress test for a setup, compilation, and execution model implementation using worst-case or malformed inputs.
20. Turn one setup, compilation, and execution model concept into a reusable utility module with header and implementation files.

Advanced coding questions

1. Implement a production-style module for setup, compilation, and execution model with header, source file, tests, and example usage.
2. Write a deliberately unsafe setup, compilation, and execution model program and then harden it against UB, leaks, and boundary bugs.
3. Create a benchmark that compares two designs for setup, compilation, and execution model and interpret the result.
4. Write a portable version of a setup, compilation, and execution model solution that avoids compiler-specific assumptions.
5. Design and document invariants for a nontrivial setup, compilation, and execution model data flow or API.
6. Make a setup, compilation, and execution model implementation pass AddressSanitizer, UndefinedBehaviorSanitizer, and warning-clean compilation.
7. Write a fuzz-friendly parser or input path involving setup, compilation, and execution model and describe what failures you expect.
8. Create a code review checklist specifically for setup, compilation, and execution model code.
9. Implement one generic abstraction involving setup, compilation, and execution model using macros, void pointers, or callbacks.
10. Write a version of a setup, compilation, and execution model module suitable for embedded constraints: fixed memory, deterministic behavior, no heap if possible.
11. Design a failure-injection test for setup, compilation, and execution model and verify cleanup correctness.

12. Write a complex bug involving setup, compilation, and execution model that only appears under optimization and explain why.
13. Implement a reusable error-reporting strategy across a setup, compilation, and execution model project.
14. Write a static-analysis-friendly version of a setup, compilation, and execution model implementation and justify design choices.
15. Model setup, compilation, and execution model ownership rules explicitly and document them in comments and README.
16. Write tests that prove the edge-case behavior of a setup, compilation, and execution model implementation.
17. Create a debugging diary for one hard setup, compilation, and execution model bug from reproduction to root cause to fix.
18. Implement a layered architecture where setup, compilation, and execution model is isolated behind a stable interface.
19. Write a version of a setup, compilation, and execution model solution that can process malformed, hostile, or oversized input safely.
20. Add assertions that enforce internal invariants in a setup, compilation, and execution model module without replacing runtime validation.
21. Demonstrate one performance pitfall in setup, compilation, and execution model and remove it with evidence.
22. Create a reusable GitHub template issue for tracking bugs in setup, compilation, and execution model code.
23. Write a setup, compilation, and execution model exercise where returning the wrong lifetime or alias creates subtle corruption, then fix it.
24. Refactor a setup, compilation, and execution model implementation so that all resources are released through a single cleanup path.
25. Write a test harness that runs many input cases automatically for setup, compilation, and execution model.
26. Implement one cross-platform consideration in setup, compilation, and execution model and document what changed.
27. Create a review comparing the naive, safe, and optimized versions of one setup, compilation, and execution model problem.
28. Write one mini project that combines setup, compilation, and execution model with at least two earlier roadmap concepts.
29. Produce an interview-grade explanation of the hardest bug class in setup, compilation, and execution model and show code.
30. Finish with a capstone for setup, compilation, and execution model that you would be proud to pin on GitHub.

2. Variables, types, operators, and conversions

Beginner coding questions

1. Write a program that declares and prints int, char, float, and double values.
2. Write a program that shows integer division versus floating-point division.
3. Write code that swaps two integers using a temporary variable.
4. Write code that swaps two integers without using a third variable and explain the risk.
5. Write a program that prints sizeof for all primitive integer types on your system.
6. Write a program that demonstrates implicit type conversion in an expression.
7. Write a program that compares signed and unsigned integers and explains the result.
8. Write code that uses every arithmetic operator at least once.
9. Write code that uses relational and logical operators to validate input.
10. Write a program that uses prefix and postfix increment and prints the difference.
11. Write a program that computes area and perimeter for a rectangle using user input.
12. Write a program that reads a char and prints its ASCII value.
13. Write a program that casts a double to int and explains truncation.
14. Write code that uses const correctly for a read-only variable.
15. Write a program that demonstrates operator precedence with and without parentheses.

Intermediate coding questions

1. Build a miniproject around variables, types, operators, and conversions that takes input, validates errors, and has at least three functions.
2. Write a broken program involving variables, types, operators, and conversions and then fix the bug with a short explanation.
3. Implement one solution for variables, types, operators, and conversions using arrays and another using pointers, then compare them.
4. Write test cases for a variables, types, operators, and conversions function covering normal, edge, and invalid input.
5. Refactor a messy variables, types, operators, and conversions program into modular .c and .h files.
6. Write code for variables, types, operators, and conversions that fails safely on invalid input instead of crashing.
7. Implement a command-line tool that demonstrates variables, types, operators, and conversions in a practical scenario.
8. Write a version of a variables, types, operators, and conversions solution with one hidden bug and challenge yourself to detect it using gdb or prints.
9. Measure the time or operation count of two variables, types, operators, and conversions approaches and explain the trade-off.

10. Write a variables, types, operators, and conversions program that avoids magic numbers by using named constants, enums, or macros.
11. Design an API for variables, types, operators, and conversions with clear ownership and error-return rules.
12. Write a program where variables, types, operators, and conversions interacts with file input or command-line input.
13. Implement boundary checks for every buffer, index, or allocation used in a variables, types, operators, and conversions exercise.
14. Create a GitHub-ready README for one variables, types, operators, and conversions exercise explaining approach, complexity, and failure cases.
15. Write a variables, types, operators, and conversions solution and then rewrite it to be more portable across compilers or platforms.
16. Introduce logging into a variables, types, operators, and conversions mini project and use it to trace execution.
17. Write a version of a variables, types, operators, and conversions exercise that handles memory cleanup through one cleanup label.
18. Create a set of interview-style oral questions from one variables, types, operators, and conversions coding problem and answer them yourself.
19. Write a stress test for a variables, types, operators, and conversions implementation using worst-case or malformed inputs.
20. Turn one variables, types, operators, and conversions concept into a reusable utility module with header and implementation files.

Advanced coding questions

1. Implement a production-style module for variables, types, operators, and conversions with header, source file, tests, and example usage.
2. Write a deliberately unsafe variables, types, operators, and conversions program and then harden it against UB, leaks, and boundary bugs.
3. Create a benchmark that compares two designs for variables, types, operators, and conversions and interpret the result.
4. Write a portable version of a variables, types, operators, and conversions solution that avoids compiler-specific assumptions.
5. Design and document invariants for a nontrivial variables, types, operators, and conversions data flow or API.
6. Make a variables, types, operators, and conversions implementation pass AddressSanitizer, UndefinedBehaviorSanitizer, and warning-clean compilation.
7. Write a fuzz-friendly parser or input path involving variables, types, operators, and conversions and describe what failures you expect.
8. Create a code review checklist specifically for variables, types, operators, and conversions code.
9. Implement one generic abstraction involving variables, types, operators, and conversions using macros, void pointers, or callbacks.

10. Write a version of a variables, types, operators, and conversions module suitable for embedded constraints: fixed memory, deterministic behavior, no heap if possible.
11. Design a failure-injection test for variables, types, operators, and conversions and verify cleanup correctness.
12. Write a complex bug involving variables, types, operators, and conversions that only appears under optimization and explain why.
13. Implement a reusable error-reporting strategy across a variables, types, operators, and conversions project.
14. Write a static-analysis-friendly version of a variables, types, operators, and conversions implementation and justify design choices.
15. Model variables, types, operators, and conversions ownership rules explicitly and document them in comments and README.
16. Write tests that prove the edge-case behavior of a variables, types, operators, and conversions implementation.
17. Create a debugging diary for one hard variables, types, operators, and conversions bug from reproduction to root cause to fix.
18. Implement a layered architecture where variables, types, operators, and conversions is isolated behind a stable interface.
19. Write a version of a variables, types, operators, and conversions solution that can process malformed, hostile, or oversized input safely.
20. Add assertions that enforce internal invariants in a variables, types, operators, and conversions module without replacing runtime validation.
21. Demonstrate one performance pitfall in variables, types, operators, and conversions and remove it with evidence.
22. Create a reusable GitHub template issue for tracking bugs in variables, types, operators, and conversions code.
23. Write a variables, types, operators, and conversions exercise where returning the wrong lifetime or alias creates subtle corruption, then fix it.
24. Refactor a variables, types, operators, and conversions implementation so that all resources are released through a single cleanup path.
25. Write a test harness that runs many input cases automatically for variables, types, operators, and conversions.
26. Implement one cross-platform consideration in variables, types, operators, and conversions and document what changed.
27. Create a review comparing the naive, safe, and optimized versions of one variables, types, operators, and conversions problem.
28. Write one mini project that combines variables, types, operators, and conversions with at least two earlier roadmap concepts.
29. Produce an interview-grade explanation of the hardest bug class in variables, types, operators, and conversions and show code.
30. Finish with a capstone for variables, types, operators, and conversions that you would be proud to pin on GitHub.

3. Input, output, and formatted text processing

Beginner coding questions

1. Write a program that reads and prints an integer using `scanf`.
2. Write a program that reads a full line using `fgets`.
3. Write a program that reads two numbers and prints their sum with `printf` formatting.
4. Write a menu-driven calculator using formatted input.
5. Write a program that safely handles invalid integer input.
6. Write a program that prints a table of numbers with aligned columns.
7. Write a program that reads name and age and prints a formatted sentence.
8. Write a program that demonstrates why `gets` is dangerous without using it.
9. Write a program that uses `sscanf` to parse values from a string.
10. Write a program that uses `snprintf` to build a message safely.
11. Write code that prints floating values with 2, 4, and 8 decimals.
12. Write a program that strips the newline from `fgets` input.
13. Write a program that reads characters until newline and counts them.
14. Write a program that echoes `stdin` until EOF.
15. Write a program that mixes `scanf` and `fgets`, then fix the newline bug.

Intermediate coding questions

1. Build a miniproject around input, output, and formatted text processing that takes input, validates errors, and has at least three functions.
2. Write a broken program involving input, output, and formatted text processing and then fix the bug with a short explanation.
3. Implement one solution for input, output, and formatted text processing using arrays and another using pointers, then compare them.
4. Write test cases for a input, output, and formatted text processing function covering normal, edge, and invalid input.
5. Refactor a messy input, output, and formatted text processing program into modular `.c` and `.h` files.
6. Write code for input, output, and formatted text processing that fails safely on invalid input instead of crashing.
7. Implement a command-line tool that demonstrates input, output, and formatted text processing in a practical scenario.
8. Write a version of a input, output, and formatted text processing solution with one hidden bug and challenge yourself to detect it using `gdb` or `prints`.
9. Measure the time or operation count of two input, output, and formatted text processing approaches and explain the trade-off.

10. Write a input, output, and formatted text processing program that avoids magic numbers by using named constants, enums, or macros.
11. Design an API for input, output, and formatted text processing with clear ownership and error-return rules.
12. Write a program where input, output, and formatted text processing interacts with file input or command-line input.
13. Implement boundary checks for every buffer, index, or allocation used in a input, output, and formatted text processing exercise.
14. Create a GitHub-ready README for one input, output, and formatted text processing exercise explaining approach, complexity, and failure cases.
15. Write a input, output, and formatted text processing solution and then rewrite it to be more portable across compilers or platforms.
16. Introduce logging into a input, output, and formatted text processing mini project and use it to trace execution.
17. Write a version of a input, output, and formatted text processing exercise that handles memory cleanup through one cleanup label.
18. Create a set of interview-style oral questions from one input, output, and formatted text processing coding problem and answer them yourself.
19. Write a stress test for a input, output, and formatted text processing implementation using worst-case or malformed inputs.
20. Turn one input, output, and formatted text processing concept into a reusable utility module with header and implementation files.

Advanced coding questions

1. Implement a production-style module for input, output, and formatted text processing with header, source file, tests, and example usage.
2. Write a deliberately unsafe input, output, and formatted text processing program and then harden it against UB, leaks, and boundary bugs.
3. Create a benchmark that compares two designs for input, output, and formatted text processing and interpret the result.
4. Write a portable version of a input, output, and formatted text processing solution that avoids compiler-specific assumptions.
5. Design and document invariants for a nontrivial input, output, and formatted text processing data flow or API.
6. Make a input, output, and formatted text processing implementation pass AddressSanitizer, UndefinedBehaviorSanitizer, and warning-clean compilation.
7. Write a fuzz-friendly parser or input path involving input, output, and formatted text processing and describe what failures you expect.
8. Create a code review checklist specifically for input, output, and formatted text processing code.
9. Implement one generic abstraction involving input, output, and formatted text processing using macros, void pointers, or callbacks.

10. Write a version of a input, output, and formatted text processing module suitable for embedded constraints: fixed memory, deterministic behavior, no heap if possible.
11. Design a failure-injection test for input, output, and formatted text processing and verify cleanup correctness.
12. Write a complex bug involving input, output, and formatted text processing that only appears under optimization and explain why.
13. Implement a reusable error-reporting strategy across a input, output, and formatted text processing project.
14. Write a static-analysis-friendly version of a input, output, and formatted text processing implementation and justify design choices.
15. Model input, output, and formatted text processing ownership rules explicitly and document them in comments and README.
16. Write tests that prove the edge-case behavior of a input, output, and formatted text processing implementation.
17. Create a debugging diary for one hard input, output, and formatted text processing bug from reproduction to root cause to fix.
18. Implement a layered architecture where input, output, and formatted text processing is isolated behind a stable interface.
19. Write a version of a input, output, and formatted text processing solution that can process malformed, hostile, or oversized input safely.
20. Add assertions that enforce internal invariants in a input, output, and formatted text processing module without replacing runtime validation.
21. Demonstrate one performance pitfall in input, output, and formatted text processing and remove it with evidence.
22. Create a reusable GitHub template issue for tracking bugs in input, output, and formatted text processing code.
23. Write a input, output, and formatted text processing exercise where returning the wrong lifetime or alias creates subtle corruption, then fix it.
24. Refactor a input, output, and formatted text processing implementation so that all resources are released through a single cleanup path.
25. Write a test harness that runs many input cases automatically for input, output, and formatted text processing.
26. Implement one cross-platform consideration in input, output, and formatted text processing and document what changed.
27. Create a review comparing the naive, safe, and optimized versions of one input, output, and formatted text processing problem.
28. Write one mini project that combines input, output, and formatted text processing with at least two earlier roadmap concepts.
29. Produce an interview-grade explanation of the hardest bug class in input, output, and formatted text processing and show code.
30. Finish with a capstone for input, output, and formatted text processing that you would be proud to pin on GitHub.

4. Control flow, functions, and recursion

Beginner coding questions

1. Write a program that checks whether a number is positive, negative, or zero.
2. Write a program that prints numbers 1 to 100 using a for loop.
3. Write a program that prints even numbers with a while loop.
4. Write a program that demonstrates break and continue.
5. Write a program that uses switch to implement a simple menu.
6. Write a function that returns the maximum of two integers.
7. Write a function that returns factorial iteratively.
8. Write a recursive factorial function.
9. Write a recursive Fibonacci function.
10. Write an iterative Fibonacci function.
11. Write a function that checks whether a number is prime.
12. Write a function that computes gcd of two numbers.
13. Write a program that counts digits of an integer.
14. Write a program that reverses an integer.
15. Write a function that prints a pattern using nested loops.

Intermediate coding questions

1. Build a miniproject around control flow, functions, and recursion that takes input, validates errors, and has at least three functions.
2. Write a broken program involving control flow, functions, and recursion and then fix the bug with a short explanation.
3. Implement one solution for control flow, functions, and recursion using arrays and another using pointers, then compare them.
4. Write test cases for a control flow, functions, and recursion function covering normal, edge, and invalid input.
5. Refactor a messy control flow, functions, and recursion program into modular .c and .h files.
6. Write code for control flow, functions, and recursion that fails safely on invalid input instead of crashing.
7. Implement a command-line tool that demonstrates control flow, functions, and recursion in a practical scenario.
8. Write a version of a control flow, functions, and recursion solution with one hidden bug and challenge yourself to detect it using gdb or prints.
9. Measure the time or operation count of two control flow, functions, and recursion approaches and explain the trade-off.
10. Write a control flow, functions, and recursion program that avoids magic numbers by using named constants, enums, or macros.

11. Design an API for control flow, functions, and recursion with clear ownership and error-return rules.
12. Write a program where control flow, functions, and recursion interacts with file input or command-line input.
13. Implement boundary checks for every buffer, index, or allocation used in a control flow, functions, and recursion exercise.
14. Create a GitHub-ready README for one control flow, functions, and recursion exercise explaining approach, complexity, and failure cases.
15. Write a control flow, functions, and recursion solution and then rewrite it to be more portable across compilers or platforms.
16. Introduce logging into a control flow, functions, and recursion mini project and use it to trace execution.
17. Write a version of a control flow, functions, and recursion exercise that handles memory cleanup through one cleanup label.
18. Create a set of interview-style oral questions from one control flow, functions, and recursion coding problem and answer them yourself.
19. Write a stress test for a control flow, functions, and recursion implementation using worst-case or malformed inputs.
20. Turn one control flow, functions, and recursion concept into a reusable utility module with header and implementation files.

Advanced coding questions

1. Implement a production-style module for control flow, functions, and recursion with header, source file, tests, and example usage.
2. Write a deliberately unsafe control flow, functions, and recursion program and then harden it against UB, leaks, and boundary bugs.
3. Create a benchmark that compares two designs for control flow, functions, and recursion and interpret the result.
4. Write a portable version of a control flow, functions, and recursion solution that avoids compiler-specific assumptions.
5. Design and document invariants for a nontrivial control flow, functions, and recursion data flow or API.
6. Make a control flow, functions, and recursion implementation pass AddressSanitizer, UndefinedBehaviorSanitizer, and warning-clean compilation.
7. Write a fuzz-friendly parser or input path involving control flow, functions, and recursion and describe what failures you expect.
8. Create a code review checklist specifically for control flow, functions, and recursion code.
9. Implement one generic abstraction involving control flow, functions, and recursion using macros, void pointers, or callbacks.
10. Write a version of a control flow, functions, and recursion module suitable for embedded constraints: fixed memory, deterministic behavior, no heap if possible.

11. Design a failure-injection test for control flow, functions, and recursion and verify cleanup correctness.
12. Write a complex bug involving control flow, functions, and recursion that only appears under optimization and explain why.
13. Implement a reusable error-reporting strategy across a control flow, functions, and recursion project.
14. Write a static-analysis-friendly version of a control flow, functions, and recursion implementation and justify design choices.
15. Model control flow, functions, and recursion ownership rules explicitly and document them in comments and README.
16. Write tests that prove the edge-case behavior of a control flow, functions, and recursion implementation.
17. Create a debugging diary for one hard control flow, functions, and recursion bug from reproduction to root cause to fix.
18. Implement a layered architecture where control flow, functions, and recursion is isolated behind a stable interface.
19. Write a version of a control flow, functions, and recursion solution that can process malformed, hostile, or oversized input safely.
20. Add assertions that enforce internal invariants in a control flow, functions, and recursion module without replacing runtime validation.
21. Demonstrate one performance pitfall in control flow, functions, and recursion and remove it with evidence.
22. Create a reusable GitHub template issue for tracking bugs in control flow, functions, and recursion code.
23. Write a control flow, functions, and recursion exercise where returning the wrong lifetime or alias creates subtle corruption, then fix it.
24. Refactor a control flow, functions, and recursion implementation so that all resources are released through a single cleanup path.
25. Write a test harness that runs many input cases automatically for control flow, functions, and recursion.
26. Implement one cross-platform consideration in control flow, functions, and recursion and document what changed.
27. Create a review comparing the naive, safe, and optimized versions of one control flow, functions, and recursion problem.
28. Write one mini project that combines control flow, functions, and recursion with at least two earlier roadmap concepts.
29. Produce an interview-grade explanation of the hardest bug class in control flow, functions, and recursion and show code.
30. Finish with a capstone for control flow, functions, and recursion that you would be proud to pin on GitHub.

5. Arrays, strings, and character handling

Beginner coding questions

1. Write a program that stores 10 integers in an array and prints them.
2. Write a program that finds the largest element in an array.
3. Write a program that reverses an array in place.
4. Write a program that copies one string into another without strcpy.
5. Write a program that calculates string length without strlen.
6. Write a program that counts vowels in a string.
7. Write a program that checks whether a string is a palindrome.
8. Write a program that concatenates two strings manually.
9. Write a program that sorts an integer array using bubble sort.
10. Write a program that reads a 3x3 matrix and prints it.
11. Write a program that adds two matrices.
12. Write a program that transposes a matrix.
13. Write a program that demonstrates array out-of-bounds and explains why it is dangerous.
14. Write a program that replaces spaces in a string with underscores.
15. Write a program that counts words in a sentence.

Intermediate coding questions

1. Build a mini project around arrays, strings, and character handling that takes input, validates errors, and has at least three functions.
2. Write a broken program involving arrays, strings, and character handling and then fix the bug with a short explanation.
3. Implement one solution for arrays, strings, and character handling using arrays and another using pointers, then compare them.
4. Write test cases for a arrays, strings, and character handling function covering normal, edge, and invalid input.
5. Refactor a messy arrays, strings, and character handling program into modular .c and .h files.
6. Write code for arrays, strings, and character handling that fails safely on invalid input instead of crashing.
7. Implement a command-line tool that demonstrates arrays, strings, and character handling in a practical scenario.
8. Write a version of a arrays, strings, and character handling solution with one hidden bug and challenge yourself to detect it using gdb or prints.
9. Measure the time or operation count of two arrays, strings, and character handling approaches and explain the trade-off.
10. Write a arrays, strings, and character handling program that avoids magic numbers by using named constants, enums, or macros.

11. Design an API for arrays, strings, and character handling with clear ownership and error-return rules.
12. Write a program where arrays, strings, and character handling interacts with file input or command-line input.
13. Implement boundary checks for every buffer, index, or allocation used in a arrays, strings, and character handling exercise.
14. Create a GitHub-ready README for one arrays, strings, and character handling exercise explaining approach, complexity, and failure cases.
15. Write a arrays, strings, and character handling solution and then rewrite it to be more portable across compilers or platforms.
16. Introduce logging into a arrays, strings, and character handling mini project and use it to trace execution.
17. Write a version of a arrays, strings, and character handling exercise that handles memory cleanup through one cleanup label.
18. Create a set of interview-style oral questions from one arrays, strings, and character handling coding problem and answer them yourself.
19. Write a stress test for a arrays, strings, and character handling implementation using worst-case or malformed inputs.
20. Turn one arrays, strings, and character handling concept into a reusable utility module with header and implementation files.

Advanced coding questions

1. Implement a production-style module for arrays, strings, and character handling with header, source file, tests, and example usage.
2. Write a deliberately unsafe arrays, strings, and character handling program and then harden it against UB, leaks, and boundary bugs.
3. Create a benchmark that compares two designs for arrays, strings, and character handling and interpret the result.
4. Write a portable version of a arrays, strings, and character handling solution that avoids compiler-specific assumptions.
5. Design and document invariants for a nontrivial arrays, strings, and character handling data flow or API.
6. Make a arrays, strings, and character handling implementation pass AddressSanitizer, UndefinedBehaviorSanitizer, and warning-clean compilation.
7. Write a fuzz-friendly parser or input path involving arrays, strings, and character handling and describe what failures you expect.
8. Create a code review checklist specifically for arrays, strings, and character handling code.
9. Implement one generic abstraction involving arrays, strings, and character handling using macros, void pointers, or callbacks.
10. Write a version of a arrays, strings, and character handling module suitable for embedded constraints: fixed memory, deterministic behavior, no heap if possible.

11. Design a failure-injection test for arrays, strings, and character handling and verify cleanup correctness.
12. Write a complex bug involving arrays, strings, and character handling that only appears under optimization and explain why.
13. Implement a reusable error-reporting strategy across a arrays, strings, and character handling project.
14. Write a static-analysis-friendly version of a arrays, strings, and character handling implementation and justify design choices.
15. Model arrays, strings, and character handling ownership rules explicitly and document them in comments and README.
16. Write tests that prove the edge-case behavior of a arrays, strings, and character handling implementation.
17. Create a debugging diary for one hard arrays, strings, and character handling bug from reproduction to root cause to fix.
18. Implement a layered architecture where arrays, strings, and character handling is isolated behind a stable interface.
19. Write a version of a arrays, strings, and character handling solution that can process malformed, hostile, or oversized input safely.
20. Add assertions that enforce internal invariants in a arrays, strings, and character handling module without replacing runtime validation.
21. Demonstrate one performance pitfall in arrays, strings, and character handling and remove it with evidence.
22. Create a reusable GitHub template issue for tracking bugs in arrays, strings, and character handling code.
23. Write a arrays, strings, and character handling exercise where returning the wrong lifetime or alias creates subtle corruption, then fix it.
24. Refactor a arrays, strings, and character handling implementation so that all resources are released through a single cleanup path.
25. Write a test harness that runs many input cases automatically for arrays, strings, and character handling.
26. Implement one cross-platform consideration in arrays, strings, and character handling and document what changed.
27. Create a review comparing the naive, safe, and optimized versions of one arrays, strings, and character handling problem.
28. Write one mini project that combines arrays, strings, and character handling with at least two earlier roadmap concepts.
29. Produce an interview-grade explanation of the hardest bug class in arrays, strings, and character handling and show code.
30. Finish with a capstone for arrays, strings, and character handling that you would be proud to pin on GitHub.

6. Pointers, const, and pointer arithmetic

Beginner coding questions

1. Write a program that prints the address of an integer variable.
2. Write a function that swaps two integers using pointers.
3. Write a program that uses a pointer to modify an array element.
4. Write a function that returns the length of a string using pointer traversal.
5. Write a program that prints array values using pointer arithmetic only.
6. Write a program that uses a pointer to pointer to modify a variable.
7. Write a function that sets a variable to zero through a pointer.
8. Write a program that demonstrates NULL checking before dereference.
9. Write code showing the difference between `int *p` and `int (*p)[5]` conceptually.
10. Write a function that receives an array and sums its elements using pointers.
11. Write a program that traverses a string with `char*`.
12. Write a program that compares two addresses from the same array.
13. Write a program that shows why returning address of a local variable is wrong.
14. Write a const-correct function that prints an integer array.
15. Write a program that uses pointer increment and decrement carefully.

Intermediate coding questions

1. Build a mini project around pointers, const, and pointer arithmetic that takes input, validates errors, and has at least three functions.
2. Write a broken program involving pointers, const, and pointer arithmetic and then fix the bug with a short explanation.
3. Implement one solution for pointers, const, and pointer arithmetic using arrays and another using pointers, then compare them.
4. Write test cases for a pointers, const, and pointer arithmetic function covering normal, edge, and invalid input.
5. Refactor a messy pointers, const, and pointer arithmetic program into modular .c and .h files.
6. Write code for pointers, const, and pointer arithmetic that fails safely on invalid input instead of crashing.
7. Implement a command-line tool that demonstrates pointers, const, and pointer arithmetic in a practical scenario.
8. Write a version of a pointers, const, and pointer arithmetic solution with one hidden bug and challenge yourself to detect it using gdb or prints.
9. Measure the time or operation count of two pointers, const, and pointer arithmetic approaches and explain the trade-off.
10. Write a pointers, const, and pointer arithmetic program that avoids magic numbers by using named constants, enums, or macros.

11. Design an API for pointers, const, and pointer arithmetic with clear ownership and error-return rules.
12. Write a program where pointers, const, and pointer arithmetic interacts with file input or command-line input.
13. Implement boundary checks for every buffer, index, or allocation used in a pointers, const, and pointer arithmetic exercise.
14. Create a GitHub-ready README for one pointers, const, and pointer arithmetic exercise explaining approach, complexity, and failure cases.
15. Write a pointers, const, and pointer arithmetic solution and then rewrite it to be more portable across compilers or platforms.
16. Introduce logging into a pointers, const, and pointer arithmetic mini project and use it to trace execution.
17. Write a version of a pointers, const, and pointer arithmetic exercise that handles memory cleanup through one cleanup label.
18. Create a set of interview-style oral questions from one pointers, const, and pointer arithmetic coding problem and answer them yourself.
19. Write a stress test for a pointers, const, and pointer arithmetic implementation using worst-case or malformed inputs.
20. Turn one pointers, const, and pointer arithmetic concept into a reusable utility module with header and implementation files.

Advanced coding questions

1. Implement a production-style module for pointers, const, and pointer arithmetic with header, source file, tests, and example usage.
2. Write a deliberately unsafe pointers, const, and pointer arithmetic program and then harden it against UB, leaks, and boundary bugs.
3. Create a benchmark that compares two designs for pointers, const, and pointer arithmetic and interpret the result.
4. Write a portable version of a pointers, const, and pointer arithmetic solution that avoids compiler-specific assumptions.
5. Design and document invariants for a nontrivial pointers, const, and pointer arithmetic data flow or API.
6. Make a pointers, const, and pointer arithmetic implementation pass AddressSanitizer, UndefinedBehaviorSanitizer, and warning-clean compilation.
7. Write a fuzz-friendly parser or input path involving pointers, const, and pointer arithmetic and describe what failures you expect.
8. Create a code review checklist specifically for pointers, const, and pointer arithmetic code.
9. Implement one generic abstraction involving pointers, const, and pointer arithmetic using macros, void pointers, or callbacks.
10. Write a version of a pointers, const, and pointer arithmetic module suitable for embedded constraints: fixed memory, deterministic behavior, no heap if possible.

11. Design a failure-injection test for pointers, const, and pointer arithmetic and verify cleanup correctness.
12. Write a complex bug involving pointers, const, and pointer arithmetic that only appears under optimization and explain why.
13. Implement a reusable error-reporting strategy across a pointers, const, and pointer arithmetic project.
14. Write a static-analysis-friendly version of a pointers, const, and pointer arithmetic implementation and justify design choices.
15. Model pointers, const, and pointer arithmetic ownership rules explicitly and document them in comments and README.
16. Write tests that prove the edge-case behavior of a pointers, const, and pointer arithmetic implementation.
17. Create a debugging diary for one hard pointers, const, and pointer arithmetic bug from reproduction to root cause to fix.
18. Implement a layered architecture where pointers, const, and pointer arithmetic is isolated behind a stable interface.
19. Write a version of a pointers, const, and pointer arithmetic solution that can process malformed, hostile, or oversized input safely.
20. Add assertions that enforce internal invariants in a pointers, const, and pointer arithmetic module without replacing runtime validation.
21. Demonstrate one performance pitfall in pointers, const, and pointer arithmetic and remove it with evidence.
22. Create a reusable GitHub template issue for tracking bugs in pointers, const, and pointer arithmetic code.
23. Write a pointers, const, and pointer arithmetic exercise where returning the wrong lifetime or alias creates subtle corruption, then fix it.
24. Refactor a pointers, const, and pointer arithmetic implementation so that all resources are released through a single cleanup path.
25. Write a test harness that runs many input cases automatically for pointers, const, and pointer arithmetic.
26. Implement one cross-platform consideration in pointers, const, and pointer arithmetic and document what changed.
27. Create a review comparing the naive, safe, and optimized versions of one pointers, const, and pointer arithmetic problem.
28. Write one mini project that combines pointers, const, and pointer arithmetic with at least two earlier roadmap concepts.
29. Produce an interview-grade explanation of the hardest bug class in pointers, const, and pointer arithmetic and show code.
30. Finish with a capstone for pointers, const, and pointer arithmetic that you would be proud to pin on GitHub.

7. Structs, unions, enums, and typedefs

Beginner coding questions

1. Define a struct for student with id, name, and marks.
2. Write a program that reads and prints one struct value.
3. Write a program with an array of structs and prints all records.
4. Write a function that takes a struct by value and prints it.
5. Write a function that takes a struct pointer and updates a field.
6. Write an enum for days of the week and print one selected day.
7. Write a union with int and float and explain the observed memory behavior.
8. Use typedef to simplify a struct declaration.
9. Write a self-referential struct for a linked-list node.
10. Write a struct containing an array field and initialize it.
11. Write code using . and -> correctly.
12. Write a struct for date and validate a simple date.
13. Write a struct for complex number and add two values.
14. Write a program to compare sizes of different structs.
15. Write a tagged enum-style state variable for a simple machine.

Intermediate coding questions

1. Build a miniproject around structs, unions, enums, and typedefs that takes input, validates errors, and has at least three functions.
2. Write a broken program involving structs, unions, enums, and typedefs and then fix the bug with a short explanation.
3. Implement one solution for structs, unions, enums, and typedefs using arrays and another using pointers, then compare them.
4. Write test cases for a structs, unions, enums, and typedefs function covering normal, edge, and invalid input.
5. Refactor a messy structs, unions, enums, and typedefs program into modular .c and .h files.
6. Write code for structs, unions, enums, and typedefs that fails safely on invalid input instead of crashing.
7. Implement a command-line tool that demonstrates structs, unions, enums, and typedefs in a practical scenario.
8. Write a version of a structs, unions, enums, and typedefs solution with one hidden bug and challenge yourself to detect it using gdb or prints.
9. Measure the time or operation count of two structs, unions, enums, and typedefs approaches and explain the trade-off.
10. Write a structs, unions, enums, and typedefs program that avoids magic numbers by using named constants, enums, or macros.

11. Design an API for structs, unions, enums, and typedefs with clear ownership and error-return rules.
12. Write a program where structs, unions, enums, and typedefs interacts with file input or command-line input.
13. Implement boundary checks for every buffer, index, or allocation used in a structs, unions, enums, and typedefs exercise.
14. Create a GitHub-ready README for one structs, unions, enums, and typedefs exercise explaining approach, complexity, and failure cases.
15. Write a structs, unions, enums, and typedefs solution and then rewrite it to be more portable across compilers or platforms.
16. Introduce logging into a structs, unions, enums, and typedefs mini project and use it to trace execution.
17. Write a version of a structs, unions, enums, and typedefs exercise that handles memory cleanup through one cleanup label.
18. Create a set of interview-style oral questions from one structs, unions, enums, and typedefs coding problem and answer them yourself.
19. Write a stress test for a structs, unions, enums, and typedefs implementation using worst-case or malformed inputs.
20. Turn one structs, unions, enums, and typedefs concept into a reusable utility module with header and implementation files.

Advanced coding questions

1. Implement a production-style module for structs, unions, enums, and typedefs with header, source file, tests, and example usage.
2. Write a deliberately unsafe structs, unions, enums, and typedefs program and then harden it against UB, leaks, and boundary bugs.
3. Create a benchmark that compares two designs for structs, unions, enums, and typedefs and interpret the result.
4. Write a portable version of a structs, unions, enums, and typedefs solution that avoids compiler-specific assumptions.
5. Design and document invariants for a nontrivial structs, unions, enums, and typedefs data flow or API.
6. Make a structs, unions, enums, and typedefs implementation pass AddressSanitizer, UndefinedBehaviorSanitizer, and warning-clean compilation.
7. Write a fuzz-friendly parser or input path involving structs, unions, enums, and typedefs and describe what failures you expect.
8. Create a code review checklist specifically for structs, unions, enums, and typedefs code.
9. Implement one generic abstraction involving structs, unions, enums, and typedefs using macros, void pointers, or callbacks.
10. Write a version of a structs, unions, enums, and typedefs module suitable for embedded constraints: fixed memory, deterministic behavior, no heap if possible.

11. Design a failure-injection test for structs, unions, enums, and typedefs and verify cleanup correctness.
12. Write a complex bug involving structs, unions, enums, and typedefs that only appears under optimization and explain why.
13. Implement a reusable error-reporting strategy across a structs, unions, enums, and typedefs project.
14. Write a static-analysis-friendly version of a structs, unions, enums, and typedefs implementation and justify design choices.
15. Model structs, unions, enums, and typedefs ownership rules explicitly and document them in comments and README.
16. Write tests that prove the edge-case behavior of a structs, unions, enums, and typedefs implementation.
17. Create a debugging diary for one hard structs, unions, enums, and typedefs bug from reproduction to root cause to fix.
18. Implement a layered architecture where structs, unions, enums, and typedefs is isolated behind a stable interface.
19. Write a version of a structs, unions, enums, and typedefs solution that can process malformed, hostile, or oversized input safely.
20. Add assertions that enforce internal invariants in a structs, unions, enums, and typedefs module without replacing runtime validation.
21. Demonstrate one performance pitfall in structs, unions, enums, and typedefs and remove it with evidence.
22. Create a reusable GitHub template issue for tracking bugs in structs, unions, enums, and typedefs code.
23. Write a structs, unions, enums, and typedefs exercise where returning the wrong lifetime or alias creates subtle corruption, then fix it.
24. Refactor a structs, unions, enums, and typedefs implementation so that all resources are released through a single cleanup path.
25. Write a test harness that runs many input cases automatically for structs, unions, enums, and typedefs.
26. Implement one cross-platform consideration in structs, unions, enums, and typedefs and document what changed.
27. Create a review comparing the naive, safe, and optimized versions of one structs, unions, enums, and typedefs problem.
28. Write one mini project that combines structs, unions, enums, and typedefs with at least two earlier roadmap concepts.
29. Produce an interview-grade explanation of the hardest bug class in structs, unions, enums, and typedefs and show code.
30. Finish with a capstone for structs, unions, enums, and typedefs that you would be proud to pin on GitHub.

8. Dynamic memory, lifetime, and ownership

Beginner coding questions

1. Allocate memory for one integer using malloc and free it.
2. Allocate memory for an array of 10 integers, fill it, and print it.
3. Use calloc to allocate zero-initialized memory and verify the contents.
4. Use realloc to grow an integer array from 5 to 10 elements.
5. Write a function that allocates a string buffer and returns it.
6. Write code that checks malloc failure before use.
7. Write a program that leaks memory intentionally, then describe the fix.
8. Write a program that frees memory exactly once.
9. Write a function that deep-copies a string using malloc.
10. Write a function that allocates a 2D matrix dynamically.
11. Free a dynamically allocated 2D matrix correctly.
12. Write a program showing difference between stack and heap arrays.
13. Write a simple dynamic array that stores integers.
14. Write a function that returns memory ownership clearly to caller.
15. Write code that sets a pointer to NULL after free and explain the limit of that habit.

Intermediate coding questions

1. Build a miniproject around dynamic memory, lifetime, and ownership that takes input, validates errors, and has at least three functions.
2. Write a broken program involving dynamic memory, lifetime, and ownership and then fix the bug with a short explanation.
3. Implement one solution for dynamic memory, lifetime, and ownership using arrays and another using pointers, then compare them.
4. Write test cases for a dynamic memory, lifetime, and ownership function covering normal, edge, and invalid input.
5. Refactor a messy dynamic memory, lifetime, and ownership program into modular .c and .h files.
6. Write code for dynamic memory, lifetime, and ownership that fails safely on invalid input instead of crashing.
7. Implement a command-line tool that demonstrates dynamic memory, lifetime, and ownership in a practical scenario.
8. Write a version of a dynamic memory, lifetime, and ownership solution with one hidden bug and challenge yourself to detect it using gdb or prints.
9. Measure the time or operation count of two dynamic memory, lifetime, and ownership approaches and explain the trade-off.

10. Write a dynamic memory, lifetime, and ownership program that avoids magic numbers by using named constants, enums, or macros.
11. Design an API for dynamic memory, lifetime, and ownership with clear ownership and error-return rules.
12. Write a program where dynamic memory, lifetime, and ownership interacts with file input or command-line input.
13. Implement boundary checks for every buffer, index, or allocation used in a dynamic memory, lifetime, and ownership exercise.
14. Create a GitHub-ready README for one dynamic memory, lifetime, and ownership exercise explaining approach, complexity, and failure cases.
15. Write a dynamic memory, lifetime, and ownership solution and then rewrite it to be more portable across compilers or platforms.
16. Introduce logging into a dynamic memory, lifetime, and ownership mini project and use it to trace execution.
17. Write a version of a dynamic memory, lifetime, and ownership exercise that handles memory cleanup through one cleanup label.
18. Create a set of interview-style oral questions from one dynamic memory, lifetime, and ownership coding problem and answer them yourself.
19. Write a stress test for a dynamic memory, lifetime, and ownership implementation using worst-case or malformed inputs.
20. Turn one dynamic memory, lifetime, and ownership concept into a reusable utility module with header and implementation files.

Advanced coding questions

1. Implement a production-style module for dynamic memory, lifetime, and ownership with header, source file, tests, and example usage.
2. Write a deliberately unsafe dynamic memory, lifetime, and ownership program and then harden it against UB, leaks, and boundary bugs.
3. Create a benchmark that compares two designs for dynamic memory, lifetime, and ownership and interpret the result.
4. Write a portable version of a dynamic memory, lifetime, and ownership solution that avoids compiler-specific assumptions.
5. Design and document invariants for a nontrivial dynamic memory, lifetime, and ownership data flow or API.
6. Make a dynamic memory, lifetime, and ownership implementation pass AddressSanitizer, UndefinedBehaviorSanitizer, and warning-clean compilation.
7. Write a fuzz-friendly parser or input path involving dynamic memory, lifetime, and ownership and describe what failures you expect.
8. Create a code review checklist specifically for dynamic memory, lifetime, and ownership code.
9. Implement one generic abstraction involving dynamic memory, lifetime, and ownership using macros, void pointers, or callbacks.

10. Write a version of a dynamic memory, lifetime, and ownership module suitable for embedded constraints: fixed memory, deterministic behavior, no heap if possible.
11. Design a failure-injection test for dynamic memory, lifetime, and ownership and verify cleanup correctness.
12. Write a complex bug involving dynamic memory, lifetime, and ownership that only appears under optimization and explain why.
13. Implement a reusable error-reporting strategy across a dynamic memory, lifetime, and ownership project.
14. Write a static-analysis-friendly version of a dynamic memory, lifetime, and ownership implementation and justify design choices.
15. Model dynamic memory, lifetime, and ownership ownership rules explicitly and document them in comments and README.
16. Write tests that prove the edge-case behavior of a dynamic memory, lifetime, and ownership implementation.
17. Create a debugging diary for one hard dynamic memory, lifetime, and ownership bug from reproduction to root cause to fix.
18. Implement a layered architecture where dynamic memory, lifetime, and ownership is isolated behind a stable interface.
19. Write a version of a dynamic memory, lifetime, and ownership solution that can process malformed, hostile, or oversized input safely.
20. Add assertions that enforce internal invariants in a dynamic memory, lifetime, and ownership module without replacing runtime validation.
21. Demonstrate one performance pitfall in dynamic memory, lifetime, and ownership and remove it with evidence.
22. Create a reusable GitHub template issue for tracking bugs in dynamic memory, lifetime, and ownership code.
23. Write a dynamic memory, lifetime, and ownership exercise where returning the wrong lifetime or alias creates subtle corruption, then fix it.
24. Refactor a dynamic memory, lifetime, and ownership implementation so that all resources are released through a single cleanup path.
25. Write a test harness that runs many input cases automatically for dynamic memory, lifetime, and ownership.
26. Implement one cross-platform consideration in dynamic memory, lifetime, and ownership and document what changed.
27. Create a review comparing the naive, safe, and optimized versions of one dynamic memory, lifetime, and ownership problem.
28. Write one mini project that combines dynamic memory, lifetime, and ownership with at least two earlier roadmap concepts.
29. Produce an interview-grade explanation of the hardest bug class in dynamic memory, lifetime, and ownership and show code.
30. Finish with a capstone for dynamic memory, lifetime, and ownership that you would be proud to pin on GitHub.

9. Preprocessor, headers, modular design, and builds

Beginner coding questions

1. Create a header with function declarations and include guards.
2. Split utility functions into `utils.c` and `utils.h`.
3. Use `#define` to declare a buffer size constant.
4. Write a function-like macro for squaring a number and test it.
5. Write the safer parenthesized version of the square macro.
6. Use `#ifdef` to compile debug-only logging.
7. Write a static helper function visible only inside one `.c` file.
8. Write an extern declaration for a global variable and use it from another file.
9. Create a multi-file calculator project.
10. Demonstrate why defining globals in headers is wrong.
11. Write a header that can be included by two `.c` files without errors.
12. Use `#if defined` to switch behavior by platform macro.
13. Write a small project and build it with a Makefile.
14. Use a macro to stringify a token.
15. Use token pasting in a tiny demo macro.

Intermediate coding questions

1. Build a miniproject around preprocessor, headers, modular design, and builds that takes input, validates errors, and has at least three functions.
2. Write a broken program involving preprocessor, headers, modular design, and builds and then fix the bug with a short explanation.
3. Implement one solution for preprocessor, headers, modular design, and builds using arrays and another using pointers, then compare them.
4. Write test cases for a preprocessor, headers, modular design, and builds function covering normal, edge, and invalid input.
5. Refactor a messy preprocessor, headers, modular design, and builds program into modular `.c` and `.h` files.
6. Write code for preprocessor, headers, modular design, and builds that fails safely on invalid input instead of crashing.
7. Implement a command-line tool that demonstrates preprocessor, headers, modular design, and builds in a practical scenario.
8. Write a version of a preprocessor, headers, modular design, and builds solution with one hidden bug and challenge yourself to detect it using `gdb` or `prints`.
9. Measure the time or operation count of two preprocessor, headers, modular design, and builds approaches and explain the trade-off.

10. Write a preprocessor, headers, modular design, and builds program that avoids magic numbers by using named constants, enums, or macros.
11. Design an API for preprocessor, headers, modular design, and builds with clear ownership and error-return rules.
12. Write a program where preprocessor, headers, modular design, and builds interacts with file input or command-line input.
13. Implement boundary checks for every buffer, index, or allocation used in a preprocessor, headers, modular design, and builds exercise.
14. Create a GitHub-ready README for one preprocessor, headers, modular design, and builds exercise explaining approach, complexity, and failure cases.
15. Write a preprocessor, headers, modular design, and builds solution and then rewrite it to be more portable across compilers or platforms.
16. Introduce logging into a preprocessor, headers, modular design, and builds mini project and use it to trace execution.
17. Write a version of a preprocessor, headers, modular design, and builds exercise that handles memory cleanup through one cleanup label.
18. Create a set of interview-style oral questions from one preprocessor, headers, modular design, and builds coding problem and answer them yourself.
19. Write a stress test for a preprocessor, headers, modular design, and builds implementation using worst-case or malformed inputs.
20. Turn one preprocessor, headers, modular design, and builds concept into a reusable utility module with header and implementation files.

Advanced coding questions

1. Implement a production-style module for preprocessor, headers, modular design, and builds with header, source file, tests, and example usage.
2. Write a deliberately unsafe preprocessor, headers, modular design, and builds program and then harden it against UB, leaks, and boundary bugs.
3. Create a benchmark that compares two designs for preprocessor, headers, modular design, and builds and interpret the result.
4. Write a portable version of a preprocessor, headers, modular design, and builds solution that avoids compiler-specific assumptions.
5. Design and document invariants for a nontrivial preprocessor, headers, modular design, and builds data flow or API.
6. Make a preprocessor, headers, modular design, and builds implementation pass AddressSanitizer, UndefinedBehaviorSanitizer, and warning-clean compilation.
7. Write a fuzz-friendly parser or input path involving preprocessor, headers, modular design, and builds and describe what failures you expect.
8. Create a code review checklist specifically for preprocessor, headers, modular design, and builds code.
9. Implement one generic abstraction involving preprocessor, headers, modular design, and builds using macros, void pointers, or callbacks.

10. Write a version of a preprocessor, headers, modular design, and builds module suitable for embedded constraints: fixed memory, deterministic behavior, no heap if possible.
11. Design a failure-injection test for preprocessor, headers, modular design, and builds and verify cleanup correctness.
12. Write a complex bug involving preprocessor, headers, modular design, and builds that only appears under optimization and explain why.
13. Implement a reusable error-reporting strategy across a preprocessor, headers, modular design, and builds project.
14. Write a static-analysis-friendly version of a preprocessor, headers, modular design, and builds implementation and justify design choices.
15. Model preprocessor, headers, modular design, and builds ownership rules explicitly and document them in comments and README.
16. Write tests that prove the edge-case behavior of a preprocessor, headers, modular design, and builds implementation.
17. Create a debugging diary for one hard preprocessor, headers, modular design, and builds bug from reproduction to root cause to fix.
18. Implement a layered architecture where preprocessor, headers, modular design, and builds is isolated behind a stable interface.
19. Write a version of a preprocessor, headers, modular design, and builds solution that can process malformed, hostile, or oversized input safely.
20. Add assertions that enforce internal invariants in a preprocessor, headers, modular design, and builds module without replacing runtime validation.
21. Demonstrate one performance pitfall in preprocessor, headers, modular design, and builds and remove it with evidence.
22. Create a reusable GitHub template issue for tracking bugs in preprocessor, headers, modular design, and builds code.
23. Write a preprocessor, headers, modular design, and builds exercise where returning the wrong lifetime or alias creates subtle corruption, then fix it.
24. Refactor a preprocessor, headers, modular design, and builds implementation so that all resources are released through a single cleanup path.
25. Write a test harness that runs many input cases automatically for preprocessor, headers, modular design, and builds.
26. Implement one cross-platform consideration in preprocessor, headers, modular design, and builds and document what changed.
27. Create a review comparing the naive, safe, and optimized versions of one preprocessor, headers, modular design, and builds problem.
28. Write one mini project that combines preprocessor, headers, modular design, and builds with at least two earlier roadmap concepts.
29. Produce an interview-grade explanation of the hardest bug class in preprocessor, headers, modular design, and builds and show code.
30. Finish with a capstone for preprocessor, headers, modular design, and builds that you would be proud to pin on GitHub.

10. File handling, binary data, and parsing

Beginner coding questions

1. Write a program that opens a text file and prints its contents line by line.
2. Write a program that counts lines, words, and characters in a file.
3. Write a program that copies one file to another.
4. Write a program that appends a line to a file.
5. Write a program that stores integers in a binary file using `fwrite`.
6. Write a program that reads integers from a binary file using `fread`.
7. Write a program that seeks to a specific byte offset.
8. Write a program that reports `fopen` failure with `perror`.
9. Write a CSV-like parser for two columns separated by comma.
10. Write a program that reads records into a struct from a text file.
11. Write a program that serializes a simple struct carefully to text.
12. Write a program that counts frequency of characters in a file.
13. Write a program that removes blank lines from a text file.
14. Write a program that merges two files into one.
15. Write a program that echoes `stdin` to a file until EOF.

Intermediate coding questions

1. Build a miniproject around file handling, binary data, and parsing that takes input, validates errors, and has at least three functions.
2. Write a broken program involving file handling, binary data, and parsing and then fix the bug with a short explanation.
3. Implement one solution for file handling, binary data, and parsing using arrays and another using pointers, then compare them.
4. Write test cases for a file handling, binary data, and parsing function covering normal, edge, and invalid input.
5. Refactor a messy file handling, binary data, and parsing program into modular `.c` and `.h` files.
6. Write code for file handling, binary data, and parsing that fails safely on invalid input instead of crashing.
7. Implement a command-line tool that demonstrates file handling, binary data, and parsing in a practical scenario.
8. Write a version of a file handling, binary data, and parsing solution with one hidden bug and challenge yourself to detect it using `gdb` or prints.
9. Measure the time or operation count of two file handling, binary data, and parsing approaches and explain the trade-off.
10. Write a file handling, binary data, and parsing program that avoids magic numbers by using named constants, enums, or macros.

11. Design an API for file handling, binary data, and parsing with clear ownership and error-return rules.
12. Write a program where file handling, binary data, and parsing interacts with file input or command-line input.
13. Implement boundary checks for every buffer, index, or allocation used in a file handling, binary data, and parsing exercise.
14. Create a GitHub-ready README for one file handling, binary data, and parsing exercise explaining approach, complexity, and failure cases.
15. Write a file handling, binary data, and parsing solution and then rewrite it to be more portable across compilers or platforms.
16. Introduce logging into a file handling, binary data, and parsing mini project and use it to trace execution.
17. Write a version of a file handling, binary data, and parsing exercise that handles memory cleanup through one cleanup label.
18. Create a set of interview-style oral questions from one file handling, binary data, and parsing coding problem and answer them yourself.
19. Write a stress test for a file handling, binary data, and parsing implementation using worst-case or malformed inputs.
20. Turn one file handling, binary data, and parsing concept into a reusable utility module with header and implementation files.

Advanced coding questions

1. Implement a production-style module for file handling, binary data, and parsing with header, source file, tests, and example usage.
2. Write a deliberately unsafe file handling, binary data, and parsing program and then harden it against UB, leaks, and boundary bugs.
3. Create a benchmark that compares two designs for file handling, binary data, and parsing and interpret the result.
4. Write a portable version of a file handling, binary data, and parsing solution that avoids compiler-specific assumptions.
5. Design and document invariants for a nontrivial file handling, binary data, and parsing data flow or API.
6. Make a file handling, binary data, and parsing implementation pass AddressSanitizer, UndefinedBehaviorSanitizer, and warning-clean compilation.
7. Write a fuzz-friendly parser or input path involving file handling, binary data, and parsing and describe what failures you expect.
8. Create a code review checklist specifically for file handling, binary data, and parsing code.
9. Implement one generic abstraction involving file handling, binary data, and parsing using macros, void pointers, or callbacks.
10. Write a version of a file handling, binary data, and parsing module suitable for embedded constraints: fixed memory, deterministic behavior, no heap if possible.

11. Design a failure-injection test for file handling, binary data, and parsing and verify cleanup correctness.
12. Write a complex bug involving file handling, binary data, and parsing that only appears under optimization and explain why.
13. Implement a reusable error-reporting strategy across a file handling, binary data, and parsing project.
14. Write a static-analysis-friendly version of a file handling, binary data, and parsing implementation and justify design choices.
15. Model file handling, binary data, and parsing ownership rules explicitly and document them in comments and README.
16. Write tests that prove the edge-case behavior of a file handling, binary data, and parsing implementation.
17. Create a debugging diary for one hard file handling, binary data, and parsing bug from reproduction to root cause to fix.
18. Implement a layered architecture where file handling, binary data, and parsing is isolated behind a stable interface.
19. Write a version of a file handling, binary data, and parsing solution that can process malformed, hostile, or oversized input safely.
20. Add assertions that enforce internal invariants in a file handling, binary data, and parsing module without replacing runtime validation.
21. Demonstrate one performance pitfall in file handling, binary data, and parsing and remove it with evidence.
22. Create a reusable GitHub template issue for tracking bugs in file handling, binary data, and parsing code.
23. Write a file handling, binary data, and parsing exercise where returning the wrong lifetime or alias creates subtle corruption, then fix it.
24. Refactor a file handling, binary data, and parsing implementation so that all resources are released through a single cleanup path.
25. Write a test harness that runs many input cases automatically for file handling, binary data, and parsing.
26. Implement one cross-platform consideration in file handling, binary data, and parsing and document what changed.
27. Create a review comparing the naive, safe, and optimized versions of one file handling, binary data, and parsing problem.
28. Write one mini project that combines file handling, binary data, and parsing with at least two earlier roadmap concepts.
29. Produce an interview-grade explanation of the hardest bug class in file handling, binary data, and parsing and show code.
30. Finish with a capstone for file handling, binary data, and parsing that you would be proud to pin on GitHub.

11. Bit manipulation, endianness, and low-level representation

Beginner coding questions

1. Write a comparator function for ascending integer sort.
2. Use `qsort` to sort an integer array.
3. Write a callback that prints each array element.
4. Write a function that accepts another function and applies it to each element.
5. Write a table of function pointers for calculator operations.
6. Write a variadic function that sums integers.
7. Write a logging function using `va_list`.
8. Write a typedef for a callback signature.
9. Write a dispatcher that calls functions based on menu choice.
10. Write a function pointer example that selects one of two algorithms.
11. Pass a context pointer to a callback and use it.
12. Write a generic swap function using `void*` and byte copying.
13. Write a generic print loop using callbacks.
14. Write a program showing what happens when function signatures do not match conceptually.
15. Write a state-machine table using function pointers.

Intermediate coding questions

1. Build a mini project around bit manipulation, endianness, and low-level representation that takes input, validates errors, and has at least three functions.
2. Write a broken program involving bit manipulation, endianness, and low-level representation and then fix the bug with a short explanation.
3. Implement one solution for bit manipulation, endianness, and low-level representation using arrays and another using pointers, then compare them.
4. Write test cases for a bit manipulation, endianness, and low-level representation function covering normal, edge, and invalid input.
5. Refactor a messy bit manipulation, endianness, and low-level representation program into modular `.c` and `.h` files.
6. Write code for bit manipulation, endianness, and low-level representation that fails safely on invalid input instead of crashing.
7. Implement a command-line tool that demonstrates bit manipulation, endianness, and low-level representation in a practical scenario.
8. Write a version of a bit manipulation, endianness, and low-level representation solution with one hidden bug and challenge yourself to detect it using `gdb` or `prints`.
9. Measure the time or operation count of two bit manipulation, endianness, and low-level representation approaches and explain the trade-off.

10. Write a bit manipulation, endianness, and low-level representation program that avoids magic numbers by using named constants, enums, or macros.
11. Design an API for bit manipulation, endianness, and low-level representation with clear ownership and error-return rules.
12. Write a program where bit manipulation, endianness, and low-level representation interacts with file input or command-line input.
13. Implement boundary checks for every buffer, index, or allocation used in a bit manipulation, endianness, and low-level representation exercise.
14. Create a GitHub-ready README for one bit manipulation, endianness, and low-level representation exercise explaining approach, complexity, and failure cases.
15. Write a bit manipulation, endianness, and low-level representation solution and then rewrite it to be more portable across compilers or platforms.
16. Introduce logging into a bit manipulation, endianness, and low-level representation mini project and use it to trace execution.
17. Write a version of a bit manipulation, endianness, and low-level representation exercise that handles memory cleanup through one cleanup label.
18. Create a set of interview-style oral questions from one bit manipulation, endianness, and low-level representation coding problem and answer them yourself.
19. Write a stress test for a bit manipulation, endianness, and low-level representation implementation using worst-case or malformed inputs.
20. Turn one bit manipulation, endianness, and low-level representation concept into a reusable utility module with header and implementation files.

Advanced coding questions

1. Implement a production-style module for bit manipulation, endianness, and low-level representation with header, source file, tests, and example usage.
2. Write a deliberately unsafe bit manipulation, endianness, and low-level representation program and then harden it against UB, leaks, and boundary bugs.
3. Create a benchmark that compares two designs for bit manipulation, endianness, and low-level representation and interpret the result.
4. Write a portable version of a bit manipulation, endianness, and low-level representation solution that avoids compiler-specific assumptions.
5. Design and document invariants for a nontrivial bit manipulation, endianness, and low-level representation data flow or API.
6. Make a bit manipulation, endianness, and low-level representation implementation pass AddressSanitizer, UndefinedBehaviorSanitizer, and warning-clean compilation.
7. Write a fuzz-friendly parser or input path involving bit manipulation, endianness, and low-level representation and describe what failures you expect.
8. Create a code review checklist specifically for bit manipulation, endianness, and low-level representation code.
9. Implement one generic abstraction involving bit manipulation, endianness, and low-level representation using macros, void pointers, or callbacks.

10. Write a version of a bit manipulation, endianness, and low-level representation module suitable for embedded constraints: fixed memory, deterministic behavior, no heap if possible.
11. Design a failure-injection test for bit manipulation, endianness, and low-level representation and verify cleanup correctness.
12. Write a complex bug involving bit manipulation, endianness, and low-level representation that only appears under optimization and explain why.
13. Implement a reusable error-reporting strategy across a bit manipulation, endianness, and low-level representation project.
14. Write a static-analysis-friendly version of a bit manipulation, endianness, and low-level representation implementation and justify design choices.
15. Model bit manipulation, endianness, and low-level representation ownership rules explicitly and document them in comments and README.
16. Write tests that prove the edge-case behavior of a bit manipulation, endianness, and low-level representation implementation.
17. Create a debugging diary for one hard bit manipulation, endianness, and low-level representation bug from reproduction to root cause to fix.
18. Implement a layered architecture where bit manipulation, endianness, and low-level representation is isolated behind a stable interface.
19. Write a version of a bit manipulation, endianness, and low-level representation solution that can process malformed, hostile, or oversized input safely.
20. Add assertions that enforce internal invariants in a bit manipulation, endianness, and low-level representation module without replacing runtime validation.
21. Demonstrate one performance pitfall in bit manipulation, endianness, and low-level representation and remove it with evidence.
22. Create a reusable GitHub template issue for tracking bugs in bit manipulation, endianness, and low-level representation code.
23. Write a bit manipulation, endianness, and low-level representation exercise where returning the wrong lifetime or alias creates subtle corruption, then fix it.
24. Refactor a bit manipulation, endianness, and low-level representation implementation so that all resources are released through a single cleanup path.
25. Write a test harness that runs many input cases automatically for bit manipulation, endianness, and low-level representation.
26. Implement one cross-platform consideration in bit manipulation, endianness, and low-level representation and document what changed.
27. Create a review comparing the naive, safe, and optimized versions of one bit manipulation, endianness, and low-level representation problem.
28. Write one mini project that combines bit manipulation, endianness, and low-level representation with at least two earlier roadmap concepts.
29. Produce an interview-grade explanation of the hardest bug class in bit manipulation, endianness, and low-level representation and show code.
30. Finish with a capstone for bit manipulation, endianness, and low-level representation that you would be proud to pin on GitHub.

12. Function pointers, callbacks, and generic programming

Beginner coding questions

1. Write code to set, clear, toggle, and test a bit in an unsigned int.
2. Write a program that prints a number in binary.
3. Write a program that checks whether a number is power of two.
4. Write code that extracts the nth bit of a number.
5. Write code that swaps odd and even bits conceptually.
6. Write a flags enum for permissions read/write/execute.
7. Write code that combines and checks permission flags.
8. Write code that packs two 4-bit values into one byte.
9. Write code that unpacks two 4-bit values from one byte.
10. Write a function that counts set bits.
11. Write a function that finds if two integers differ by exactly one bit.
12. Write a program that demonstrates left and right shifts.
13. Write code that detects endian order at runtime.
14. Write code that converts a 16-bit value to big-endian byte order manually.
15. Write a register-style mask and field extraction example.

Intermediate coding questions

1. Build a miniproject around function pointers, callbacks, and generic programming that takes input, validates errors, and has at least three functions.
2. Write a broken program involving function pointers, callbacks, and generic programming and then fix the bug with a short explanation.
3. Implement one solution for function pointers, callbacks, and generic programming using arrays and another using pointers, then compare them.
4. Write test cases for a function pointers, callbacks, and generic programming function covering normal, edge, and invalid input.
5. Refactor a messy function pointers, callbacks, and generic programming program into modular .c and .h files.
6. Write code for function pointers, callbacks, and generic programming that fails safely on invalid input instead of crashing.
7. Implement a command-line tool that demonstrates function pointers, callbacks, and generic programming in a practical scenario.
8. Write a version of a function pointers, callbacks, and generic programming solution with one hidden bug and challenge yourself to detect it using gdb or prints.
9. Measure the time or operation count of two function pointers, callbacks, and generic programming approaches and explain the trade-off.

10. Write a function pointers, callbacks, and generic programming program that avoids magic numbers by using named constants, enums, or macros.
11. Design an API for function pointers, callbacks, and generic programming with clear ownership and error-return rules.
12. Write a program where function pointers, callbacks, and generic programming interacts with file input or command-line input.
13. Implement boundary checks for every buffer, index, or allocation used in a function pointers, callbacks, and generic programming exercise.
14. Create a GitHub-ready README for one function pointers, callbacks, and generic programming exercise explaining approach, complexity, and failure cases.
15. Write a function pointers, callbacks, and generic programming solution and then rewrite it to be more portable across compilers or platforms.
16. Introduce logging into a function pointers, callbacks, and generic programming mini project and use it to trace execution.
17. Write a version of a function pointers, callbacks, and generic programming exercise that handles memory cleanup through one cleanup label.
18. Create a set of interview-style oral questions from one function pointers, callbacks, and generic programming coding problem and answer them yourself.
19. Write a stress test for a function pointers, callbacks, and generic programming implementation using worst-case or malformed inputs.
20. Turn one function pointers, callbacks, and generic programming concept into a reusable utility module with header and implementation files.

Advanced coding questions

1. Implement a production-style module for function pointers, callbacks, and generic programming with header, source file, tests, and example usage.
2. Write a deliberately unsafe function pointers, callbacks, and generic programming program and then harden it against UB, leaks, and boundary bugs.
3. Create a benchmark that compares two designs for function pointers, callbacks, and generic programming and interpret the result.
4. Write a portable version of a function pointers, callbacks, and generic programming solution that avoids compiler-specific assumptions.
5. Design and document invariants for a nontrivial function pointers, callbacks, and generic programming data flow or API.
6. Make a function pointers, callbacks, and generic programming implementation pass AddressSanitizer, UndefinedBehaviorSanitizer, and warning-clean compilation.
7. Write a fuzz-friendly parser or input path involving function pointers, callbacks, and generic programming and describe what failures you expect.
8. Create a code review checklist specifically for function pointers, callbacks, and generic programming code.
9. Implement one generic abstraction involving function pointers, callbacks, and generic programming using macros, void pointers, or callbacks.

10. Write a version of a function pointers, callbacks, and generic programming module suitable for embedded constraints: fixed memory, deterministic behavior, no heap if possible.
11. Design a failure-injection test for function pointers, callbacks, and generic programming and verify cleanup correctness.
12. Write a complex bug involving function pointers, callbacks, and generic programming that only appears under optimization and explain why.
13. Implement a reusable error-reporting strategy across a function pointers, callbacks, and generic programming project.
14. Write a static-analysis-friendly version of a function pointers, callbacks, and generic programming implementation and justify design choices.
15. Model function pointers, callbacks, and generic programming ownership rules explicitly and document them in comments and README.
16. Write tests that prove the edge-case behavior of a function pointers, callbacks, and generic programming implementation.
17. Create a debugging diary for one hard function pointers, callbacks, and generic programming bug from reproduction to root cause to fix.
18. Implement a layered architecture where function pointers, callbacks, and generic programming is isolated behind a stable interface.
19. Write a version of a function pointers, callbacks, and generic programming solution that can process malformed, hostile, or oversized input safely.
20. Add assertions that enforce internal invariants in a function pointers, callbacks, and generic programming module without replacing runtime validation.
21. Demonstrate one performance pitfall in function pointers, callbacks, and generic programming and remove it with evidence.
22. Create a reusable GitHub template issue for tracking bugs in function pointers, callbacks, and generic programming code.
23. Write a function pointers, callbacks, and generic programming exercise where returning the wrong lifetime or alias creates subtle corruption, then fix it.
24. Refactor a function pointers, callbacks, and generic programming implementation so that all resources are released through a single cleanup path.
25. Write a test harness that runs many input cases automatically for function pointers, callbacks, and generic programming.
26. Implement one cross-platform consideration in function pointers, callbacks, and generic programming and document what changed.
27. Create a review comparing the naive, safe, and optimized versions of one function pointers, callbacks, and generic programming problem.
28. Write one mini project that combines function pointers, callbacks, and generic programming with at least two earlier roadmap concepts.
29. Produce an interview-grade explanation of the hardest bug class in function pointers, callbacks, and generic programming and show code.
30. Finish with a capstone for function pointers, callbacks, and generic programming that you would be proud to pin on GitHub.

13. Data structures in C

Beginner coding questions

1. Implement a singly linked list insertion at head.
2. Implement linked list insertion at tail.
3. Implement linked list deletion by value.
4. Implement stack using array.
5. Implement queue using array.
6. Implement queue using linked list.
7. Implement a simple dynamic vector of ints.
8. Implement linear search on an array.
9. Implement binary search on a sorted array.
10. Implement a linked list traversal printer.
11. Implement a function that reverses a linked list.
12. Implement a node count function for linked list.
13. Implement a simple hash table with very small fixed bucket count.
14. Implement push, pop, and peek for stack.
15. Implement enqueue, dequeue, and front for queue.

Intermediate coding questions

1. Build a mini project around data structures in c that takes input, validates errors, and has at least three functions.
2. Write a broken program involving data structures in c and then fix the bug with a short explanation.
3. Implement one solution for data structures in c using arrays and another using pointers, then compare them.
4. Write test cases for a data structures in c function covering normal, edge, and invalid input.
5. Refactor a messy data structures in c program into modular .c and .h files.
6. Write code for data structures in c that fails safely on invalid input instead of crashing.
7. Implement a command-line tool that demonstrates data structures in c in a practical scenario.
8. Write a version of a data structures in c solution with one hidden bug and challenge yourself to detect it using gdb or prints.
9. Measure the time or operation count of two data structures in c approaches and explain the trade-off.
10. Write a data structures in c program that avoids magic numbers by using named constants, enums, or macros.
11. Design an API for data structures in c with clear ownership and error-return rules.
12. Write a program where data structures in c interacts with file input or command-line input.

13. Implement boundary checks for every buffer, index, or allocation used in a data structures in c exercise.
14. Create a GitHub-ready README for one data structures in c exercise explaining approach, complexity, and failure cases.
15. Write a data structures in c solution and then rewrite it to be more portable across compilers or platforms.
16. Introduce logging into a data structures in c mini project and use it to trace execution.
17. Write a version of a data structures in c exercise that handles memory cleanup through one cleanup label.
18. Create a set of interview-style oral questions from one data structures in c coding problem and answer them yourself.
19. Write a stress test for a data structures in c implementation using worst-case or malformed inputs.
20. Turn one data structures in c concept into a reusable utility module with header and implementation files.

Advanced coding questions

1. Implement a production-style module for data structures in c with header, source file, tests, and example usage.
2. Write a deliberately unsafe data structures in c program and then harden it against UB, leaks, and boundary bugs.
3. Create a benchmark that compares two designs for data structures in c and interpret the result.
4. Write a portable version of a data structures in c solution that avoids compiler-specific assumptions.
5. Design and document invariants for a nontrivial data structures in c data flow or API.
6. Make a data structures in c implementation pass AddressSanitizer, UndefinedBehaviorSanitizer, and warning-clean compilation.
7. Write a fuzz-friendly parser or input path involving data structures in c and describe what failures you expect.
8. Create a code review checklist specifically for data structures in c code.
9. Implement one generic abstraction involving data structures in c using macros, void pointers, or callbacks.
10. Write a version of a data structures in c module suitable for embedded constraints: fixed memory, deterministic behavior, no heap if possible.
11. Design a failure-injection test for data structures in c and verify cleanup correctness.
12. Write a complex bug involving data structures in c that only appears under optimization and explain why.
13. Implement a reusable error-reporting strategy across a data structures in c project.
14. Write a static-analysis-friendly version of a data structures in c implementation and justify design choices.

15. Model data structures in c ownership rules explicitly and document them in comments and README.
16. Write tests that prove the edge-case behavior of a data structures in c implementation.
17. Create a debugging diary for one hard data structures in c bug from reproduction to root cause to fix.
18. Implement a layered architecture where data structures in c is isolated behind a stable interface.
19. Write a version of a data structures in c solution that can process malformed, hostile, or oversized input safely.
20. Add assertions that enforce internal invariants in a data structures in c module without replacing runtime validation.
21. Demonstrate one performance pitfall in data structures in c and remove it with evidence.
22. Create a reusable GitHub template issue for tracking bugs in data structures in c code.
23. Write a data structures in c exercise where returning the wrong lifetime or alias creates subtle corruption, then fix it.
24. Refactor a data structures in c implementation so that all resources are released through a single cleanup path.
25. Write a test harness that runs many input cases automatically for data structures in c.
26. Implement one cross-platform consideration in data structures in c and document what changed.
27. Create a review comparing the naive, safe, and optimized versions of one data structures in c problem.
28. Write one mini project that combines data structures in c with at least two earlier roadmap concepts.
29. Produce an interview-grade explanation of the hardest bug class in data structures in c and show code.
30. Finish with a capstone for data structures in c that you would be proud to pin on GitHub.

14. Algorithms, complexity, and problem solving in C

Beginner coding questions

1. Implement bubble sort for integers.
2. Implement selection sort for integers.
3. Implement insertion sort for integers.
4. Implement merge step of merge sort.
5. Implement recursive binary search.
6. Write a program to find second largest element in an array.
7. Write a program to remove duplicates from a sorted array.
8. Write a program to rotate an array by k positions.
9. Write a program to find missing number from 1 to n.

10. Write a program to find duplicate number in a constrained array.
11. Write a function to check balanced parentheses using stack.
12. Write a function to evaluate postfix expression.
13. Write a greedy coin-change demo and explain when it fails.
14. Write a function to merge two sorted arrays.
15. Write a function to detect cycle in linked list using two pointers.

Intermediate coding questions

1. Build a mini project around algorithms, complexity, and problem solving in c that takes input, validates errors, and has at least three functions.
2. Write a broken program involving algorithms, complexity, and problem solving in c and then fix the bug with a short explanation.
3. Implement one solution for algorithms, complexity, and problem solving in c using arrays and another using pointers, then compare them.
4. Write test cases for a algorithms, complexity, and problem solving in c function covering normal, edge, and invalid input.
5. Refactor a messy algorithms, complexity, and problem solving in c program into modular .c and .h files.
6. Write code for algorithms, complexity, and problem solving in c that fails safely on invalid input instead of crashing.
7. Implement a command-line tool that demonstrates algorithms, complexity, and problem solving in c in a practical scenario.
8. Write a version of a algorithms, complexity, and problem solving in c solution with one hidden bug and challenge yourself to detect it using gdb or prints.
9. Measure the time or operation count of two algorithms, complexity, and problem solving in c approaches and explain the trade-off.
10. Write a algorithms, complexity, and problem solving in c program that avoids magic numbers by using named constants, enums, or macros.
11. Design an API for algorithms, complexity, and problem solving in c with clear ownership and error-return rules.
12. Write a program where algorithms, complexity, and problem solving in c interacts with file input or command-line input.
13. Implement boundary checks for every buffer, index, or allocation used in a algorithms, complexity, and problem solving in c exercise.
14. Create a GitHub-ready README for one algorithms, complexity, and problem solving in c exercise explaining approach, complexity, and failure cases.
15. Write a algorithms, complexity, and problem solving in c solution and then rewrite it to be more portable across compilers or platforms.
16. Introduce logging into a algorithms, complexity, and problem solving in c mini project and use it to trace execution.

17. Write a version of a algorithms, complexity, and problem solving in c exercise that handles memory cleanup through one cleanup label.
18. Create a set of interview-style oral questions from one algorithms, complexity, and problem solving in c coding problem and answer them yourself.
19. Write a stress test for a algorithms, complexity, and problem solving in c implementation using worst-case or malformed inputs.
20. Turn one algorithms, complexity, and problem solving in c concept into a reusable utility module with header and implementation files.

Advanced coding questions

1. Implement a production-style module for algorithms, complexity, and problem solving in c with header, source file, tests, and example usage.
2. Write a deliberately unsafe algorithms, complexity, and problem solving in c program and then harden it against UB, leaks, and boundary bugs.
3. Create a benchmark that compares two designs for algorithms, complexity, and problem solving in c and interpret the result.
4. Write a portable version of a algorithms, complexity, and problem solving in c solution that avoids compiler-specific assumptions.
5. Design and document invariants for a nontrivial algorithms, complexity, and problem solving in c data flow or API.
6. Make a algorithms, complexity, and problem solving in c implementation pass AddressSanitizer, UndefinedBehaviorSanitizer, and warning-clean compilation.
7. Write a fuzz-friendly parser or input path involving algorithms, complexity, and problem solving in c and describe what failures you expect.
8. Create a code review checklist specifically for algorithms, complexity, and problem solving in c code.
9. Implement one generic abstraction involving algorithms, complexity, and problem solving in c using macros, void pointers, or callbacks.
10. Write a version of a algorithms, complexity, and problem solving in c module suitable for embedded constraints: fixed memory, deterministic behavior, no heap if possible.
11. Design a failure-injection test for algorithms, complexity, and problem solving in c and verify cleanup correctness.
12. Write a complex bug involving algorithms, complexity, and problem solving in c that only appears under optimization and explain why.
13. Implement a reusable error-reporting strategy across a algorithms, complexity, and problem solving in c project.
14. Write a static-analysis-friendly version of a algorithms, complexity, and problem solving in c implementation and justify design choices.
15. Model algorithms, complexity, and problem solving in c ownership rules explicitly and document them in comments and README.
16. Write tests that prove the edge-case behavior of a algorithms, complexity, and problem solving in c implementation.

17. Create a debugging diary for one hard algorithms, complexity, and problem solving in c bug from reproduction to root cause to fix.
18. Implement a layered architecture where algorithms, complexity, and problem solving in c is isolated behind a stable interface.
19. Write a version of a algorithms, complexity, and problem solving in c solution that can process malformed, hostile, or oversized input safely.
20. Add assertions that enforce internal invariants in a algorithms, complexity, and problem solving in c module without replacing runtime validation.
21. Demonstrate one performance pitfall in algorithms, complexity, and problem solving in c and remove it with evidence.
22. Create a reusable GitHub template issue for tracking bugs in algorithms, complexity, and problem solving in c code.
23. Write a algorithms, complexity, and problem solving in c exercise where returning the wrong lifetime or alias creates subtle corruption, then fix it.
24. Refactor a algorithms, complexity, and problem solving in c implementation so that all resources are released through a single cleanup path.
25. Write a test harness that runs many input cases automatically for algorithms, complexity, and problem solving in c.
26. Implement one cross-platform consideration in algorithms, complexity, and problem solving in c and document what changed.
27. Create a review comparing the naive, safe, and optimized versions of one algorithms, complexity, and problem solving in c problem.
28. Write one mini project that combines algorithms, complexity, and problem solving in c with at least two earlier roadmap concepts.
29. Produce an interview-grade explanation of the hardest bug class in algorithms, complexity, and problem solving in c and show code.
30. Finish with a capstone for algorithms, complexity, and problem solving in c that you would be proud to pin on GitHub.

15. Debugging, testing, sanitizers, and tooling

Beginner coding questions

1. Compile a program with -Wall -Wextra -Werror and fix every issue.
2. Use gdb to stop at main and inspect a variable.
3. Write a program with a buffer overflow and detect it with AddressSanitizer.
4. Write a program with a memory leak and inspect it with valgrind.
5. Write a tiny assert-based test for a max() function.
6. Write tests for a string reverse function.
7. Create a failing test before fixing a bug in array code.
8. Use printf debugging to isolate a loop-boundary bug.

9. Write a script or Makefile target for debug build and release build.
10. Write code that triggers an undefined behavior sanitizer warning.
11. Use gcov or similar tooling conceptually on a tiny program.
12. Write a reproducible bug report for a pointer bug.
13. Create unit tests for a linked list insert function.
14. Profile a loop-heavy program and compare two versions.
15. Add defensive asserts to a small data-structure implementation.

Intermediate coding questions

1. Build a mini project around debugging, testing, sanitizers, and tooling that takes input, validates errors, and has at least three functions.
2. Write a broken program involving debugging, testing, sanitizers, and tooling and then fix the bug with a short explanation.
3. Implement one solution for debugging, testing, sanitizers, and tooling using arrays and another using pointers, then compare them.
4. Write test cases for a debugging, testing, sanitizers, and tooling function covering normal, edge, and invalid input.
5. Refactor a messy debugging, testing, sanitizers, and tooling program into modular .c and .h files.
6. Write code for debugging, testing, sanitizers, and tooling that fails safely on invalid input instead of crashing.
7. Implement a command-line tool that demonstrates debugging, testing, sanitizers, and tooling in a practical scenario.
8. Write a version of a debugging, testing, sanitizers, and tooling solution with one hidden bug and challenge yourself to detect it using gdb or prints.
9. Measure the time or operation count of two debugging, testing, sanitizers, and tooling approaches and explain the trade-off.
10. Write a debugging, testing, sanitizers, and tooling program that avoids magic numbers by using named constants, enums, or macros.
11. Design an API for debugging, testing, sanitizers, and tooling with clear ownership and error-return rules.
12. Write a program where debugging, testing, sanitizers, and tooling interacts with file input or command-line input.
13. Implement boundary checks for every buffer, index, or allocation used in a debugging, testing, sanitizers, and tooling exercise.
14. Create a GitHub-ready README for one debugging, testing, sanitizers, and tooling exercise explaining approach, complexity, and failure cases.
15. Write a debugging, testing, sanitizers, and tooling solution and then rewrite it to be more portable across compilers or platforms.
16. Introduce logging into a debugging, testing, sanitizers, and tooling mini project and use it to trace execution.

17. Write a version of a debugging, testing, sanitizers, and tooling exercise that handles memory cleanup through one cleanup label.
18. Create a set of interview-style oral questions from one debugging, testing, sanitizers, and tooling coding problem and answer them yourself.
19. Write a stress test for a debugging, testing, sanitizers, and tooling implementation using worst-case or malformed inputs.
20. Turn one debugging, testing, sanitizers, and tooling concept into a reusable utility module with header and implementation files.

Advanced coding questions

1. Implement a production-style module for debugging, testing, sanitizers, and tooling with header, source file, tests, and example usage.
2. Write a deliberately unsafe debugging, testing, sanitizers, and tooling program and then harden it against UB, leaks, and boundary bugs.
3. Create a benchmark that compares two designs for debugging, testing, sanitizers, and tooling and interpret the result.
4. Write a portable version of a debugging, testing, sanitizers, and tooling solution that avoids compiler-specific assumptions.
5. Design and document invariants for a nontrivial debugging, testing, sanitizers, and tooling data flow or API.
6. Make a debugging, testing, sanitizers, and tooling implementation pass AddressSanitizer, UndefinedBehaviorSanitizer, and warning-clean compilation.
7. Write a fuzz-friendly parser or input path involving debugging, testing, sanitizers, and tooling and describe what failures you expect.
8. Create a code review checklist specifically for debugging, testing, sanitizers, and tooling code.
9. Implement one generic abstraction involving debugging, testing, sanitizers, and tooling using macros, void pointers, or callbacks.
10. Write a version of a debugging, testing, sanitizers, and tooling module suitable for embedded constraints: fixed memory, deterministic behavior, no heap if possible.
11. Design a failure-injection test for debugging, testing, sanitizers, and tooling and verify cleanup correctness.
12. Write a complex bug involving debugging, testing, sanitizers, and tooling that only appears under optimization and explain why.
13. Implement a reusable error-reporting strategy across a debugging, testing, sanitizers, and tooling project.
14. Write a static-analysis-friendly version of a debugging, testing, sanitizers, and tooling implementation and justify design choices.
15. Model debugging, testing, sanitizers, and tooling ownership rules explicitly and document them in comments and README.
16. Write tests that prove the edge-case behavior of a debugging, testing, sanitizers, and tooling implementation.

17. Create a debugging diary for one hard debugging, testing, sanitizers, and tooling bug from reproduction to root cause to fix.
18. Implement a layered architecture where debugging, testing, sanitizers, and tooling is isolated behind a stable interface.
19. Write a version of a debugging, testing, sanitizers, and tooling solution that can process malformed, hostile, or oversized input safely.
20. Add assertions that enforce internal invariants in a debugging, testing, sanitizers, and tooling module without replacing runtime validation.
21. Demonstrate one performance pitfall in debugging, testing, sanitizers, and tooling and remove it with evidence.
22. Create a reusable GitHub template issue for tracking bugs in debugging, testing, sanitizers, and tooling code.
23. Write a debugging, testing, sanitizers, and tooling exercise where returning the wrong lifetime or alias creates subtle corruption, then fix it.
24. Refactor a debugging, testing, sanitizers, and tooling implementation so that all resources are released through a single cleanup path.
25. Write a test harness that runs many input cases automatically for debugging, testing, sanitizers, and tooling.
26. Implement one cross-platform consideration in debugging, testing, sanitizers, and tooling and document what changed.
27. Create a review comparing the naive, safe, and optimized versions of one debugging, testing, sanitizers, and tooling problem.
28. Write one mini project that combines debugging, testing, sanitizers, and tooling with at least two earlier roadmap concepts.
29. Produce an interview-grade explanation of the hardest bug class in debugging, testing, sanitizers, and tooling and show code.
30. Finish with a capstone for debugging, testing, sanitizers, and tooling that you would be proud to pin on GitHub.

16. System programming, concurrency, embedded C, and expert-level pitfalls

Beginner coding questions

1. Write a CLI tool that reads command-line arguments and prints parsed options.
2. Write a program that reads an environment variable and falls back to a default.
3. Write a POSIX-style program that creates a child process conceptually or with fork if available.
4. Write a threaded counter with one mutex protecting shared state.
5. Write a producer-consumer demo with a fixed-size buffer conceptually or with pthreads.
6. Write code using volatile for a mock memory-mapped register variable and explain what it does not guarantee.
7. Write a bit-mask driven register configuration demo for an embedded peripheral.
8. Write a ring buffer for UART-style data handling.

9. Write a simple event loop that dispatches function callbacks.
10. Write a state machine for an embedded button debounce scenario.
11. Write code that simulates an interrupt flag checked by the main loop.
12. Write a threaded bug intentionally causing race condition, then fix it.
13. Write a program that demonstrates deadlock with two mutexes, then fix lock ordering.
14. Write a signal-handling example that sets a flag and exits cleanly.
15. Write a small project that combines file I/O, callbacks, dynamic memory, and error handling.

Intermediate coding questions

1. Build a mini project around system programming, concurrency, embedded c, and expert-level pitfalls that takes input, validates errors, and has at least three functions.
2. Write a broken program involving system programming, concurrency, embedded c, and expert-level pitfalls and then fix the bug with a short explanation.
3. Implement one solution for system programming, concurrency, embedded c, and expert-level pitfalls using arrays and another using pointers, then compare them.
4. Write test cases for a system programming, concurrency, embedded c, and expert-level pitfalls function covering normal, edge, and invalid input.
5. Refactor a messy system programming, concurrency, embedded c, and expert-level pitfalls program into modular .c and .h files.
6. Write code for system programming, concurrency, embedded c, and expert-level pitfalls that fails safely on invalid input instead of crashing.
7. Implement a command-line tool that demonstrates system programming, concurrency, embedded c, and expert-level pitfalls in a practical scenario.
8. Write a version of a system programming, concurrency, embedded c, and expert-level pitfalls solution with one hidden bug and challenge yourself to detect it using gdb or prints.
9. Measure the time or operation count of two system programming, concurrency, embedded c, and expert-level pitfalls approaches and explain the trade-off.
10. Write a system programming, concurrency, embedded c, and expert-level pitfalls program that avoids magic numbers by using named constants, enums, or macros.
11. Design an API for system programming, concurrency, embedded c, and expert-level pitfalls with clear ownership and error-return rules.
12. Write a program where system programming, concurrency, embedded c, and expert-level pitfalls interacts with file input or command-line input.
13. Implement boundary checks for every buffer, index, or allocation used in a system programming, concurrency, embedded c, and expert-level pitfalls exercise.
14. Create a GitHub-ready README for one system programming, concurrency, embedded c, and expert-level pitfalls exercise explaining approach, complexity, and failure cases.
15. Write a system programming, concurrency, embedded c, and expert-level pitfalls solution and then rewrite it to be more portable across compilers or platforms.
16. Introduce logging into a system programming, concurrency, embedded c, and expert-level pitfalls mini project and use it to trace execution.

17. Write a version of a system programming, concurrency, embedded c, and expert-level pitfalls exercise that handles memory cleanup through one cleanup label.
18. Create a set of interview-style oral questions from one system programming, concurrency, embedded c, and expert-level pitfalls coding problem and answer them yourself.
19. Write a stress test for a system programming, concurrency, embedded c, and expert-level pitfalls implementation using worst-case or malformed inputs.
20. Turn one system programming, concurrency, embedded c, and expert-level pitfalls concept into a reusable utility module with header and implementation files.

Advanced coding questions

1. Implement a production-style module for system programming, concurrency, embedded c, and expert-level pitfalls with header, source file, tests, and example usage.
2. Write a deliberately unsafe system programming, concurrency, embedded c, and expert-level pitfalls program and then harden it against UB, leaks, and boundary bugs.
3. Create a benchmark that compares two designs for system programming, concurrency, embedded c, and expert-level pitfalls and interpret the result.
4. Write a portable version of a system programming, concurrency, embedded c, and expert-level pitfalls solution that avoids compiler-specific assumptions.
5. Design and document invariants for a nontrivial system programming, concurrency, embedded c, and expert-level pitfalls data flow or API.
6. Make a system programming, concurrency, embedded c, and expert-level pitfalls implementation pass AddressSanitizer, UndefinedBehaviorSanitizer, and warning-clean compilation.
7. Write a fuzz-friendly parser or input path involving system programming, concurrency, embedded c, and expert-level pitfalls and describe what failures you expect.
8. Create a code review checklist specifically for system programming, concurrency, embedded c, and expert-level pitfalls code.
9. Implement one generic abstraction involving system programming, concurrency, embedded c, and expert-level pitfalls using macros, void pointers, or callbacks.
10. Write a version of a system programming, concurrency, embedded c, and expert-level pitfalls module suitable for embedded constraints: fixed memory, deterministic behavior, no heap if possible.
11. Design a failure-injection test for system programming, concurrency, embedded c, and expert-level pitfalls and verify cleanup correctness.
12. Write a complex bug involving system programming, concurrency, embedded c, and expert-level pitfalls that only appears under optimization and explain why.
13. Implement a reusable error-reporting strategy across a system programming, concurrency, embedded c, and expert-level pitfalls project.
14. Write a static-analysis-friendly version of a system programming, concurrency, embedded c, and expert-level pitfalls implementation and justify design choices.
15. Model system programming, concurrency, embedded c, and expert-level pitfalls ownership rules explicitly and document them in comments and README.

16. Write tests that prove the edge-case behavior of a system programming, concurrency, embedded c, and expert-level pitfalls implementation.
17. Create a debugging diary for one hard system programming, concurrency, embedded c, and expert-level pitfalls bug from reproduction to root cause to fix.
18. Implement a layered architecture where system programming, concurrency, embedded c, and expert-level pitfalls is isolated behind a stable interface.
19. Write a version of a system programming, concurrency, embedded c, and expert-level pitfalls solution that can process malformed, hostile, or oversized input safely.
20. Add assertions that enforce internal invariants in a system programming, concurrency, embedded c, and expert-level pitfalls module without replacing runtime validation.
21. Demonstrate one performance pitfall in system programming, concurrency, embedded c, and expert-level pitfalls and remove it with evidence.
22. Create a reusable GitHub template issue for tracking bugs in system programming, concurrency, embedded c, and expert-level pitfalls code.
23. Write a system programming, concurrency, embedded c, and expert-level pitfalls exercise where returning the wrong lifetime or alias creates subtle corruption, then fix it.
24. Refactor a system programming, concurrency, embedded c, and expert-level pitfalls implementation so that all resources are released through a single cleanup path.
25. Write a test harness that runs many input cases automatically for system programming, concurrency, embedded c, and expert-level pitfalls.
26. Implement one cross-platform consideration in system programming, concurrency, embedded c, and expert-level pitfalls and document what changed.
27. Create a review comparing the naive, safe, and optimized versions of one system programming, concurrency, embedded c, and expert-level pitfalls problem.
28. Write one mini project that combines system programming, concurrency, embedded c, and expert-level pitfalls with at least two earlier roadmap concepts.
29. Produce an interview-grade explanation of the hardest bug class in system programming, concurrency, embedded c, and expert-level pitfalls and show code.
30. Finish with a capstone for system programming, concurrency, embedded c, and expert-level pitfalls that you would be proud to pin on GitHub.

Notion-style tracker page content

Copy the following table structure into Notion, GitHub Projects, or Markdown:

Concept	Status	Repo Link	Notes	Beginner Done	Intermediate Done	Advanced Done	Mini Project	Bugs Found	Revision Score
Setup, compilation, and execution model	Not started			0/15	0/20	0/30			0
Variables, types, operators, and conversions	Not started			0/15	0/20	0/30			0

Concept	Status	Repo Link	Notes	Beginner Done	Intermediate Done	Advanced Done	Mini Project	Bugs Found	Revision Score
Input, output, and formatted text processing	Not started			0/15	0/20	0/30			0
Control flow, functions, and recursion	Not started			0/15	0/20	0/30			0
Arrays, strings, and character handling	Not started			0/15	0/20	0/30			0
Pointers, const, and pointer arithmetic	Not started			0/15	0/20	0/30			0
Structs, unions, enums, and typedefs	Not started			0/15	0/20	0/30			0
Dynamic memory, lifetime, and ownership	Not started			0/15	0/20	0/30			0
Preprocessor, headers, modular design, and builds	Not started			0/15	0/20	0/30			0
File handling, binary data, and parsing	Not started			0/15	0/20	0/30			0
Bit manipulation, endianness, and low-level representation	Not started			0/15	0/20	0/30			0
Function pointers, callbacks, and generic programming	Not started			0/15	0/20	0/30			0
Data structures in C	Not started			0/15	0/20	0/30			0
Algorithms, complexity, and problem solving in C	Not started			0/15	0/20	0/30			0

Concept	Status	Repo Link	Notes	Beginner Done	Intermediate Done	Advanced Done	Mini Project	Bugs Found	Revision Score
Debugging, testing, sanitizers, and tooling	Not started			0/15	0/20	0/30			0
System programming, concurrency, embedded C, and expert-level pitfalls	Not started			0/15	0/20	0/30			0

GitHub folder structure

```

c-mastery-v2/
  01-setup-compilation-and-execution-model/
    README.md
    notes.md
    mistakes.md
    questions.md
    src/
    tests/
  02-variables-types-operators-and-conversions/
    README.md
    notes.md
    mistakes.md
    questions.md
    src/
    tests/
  03-input-output-and-formatted-text-processing/
    README.md
    notes.md
    mistakes.md
    questions.md
    src/
    tests/
  04-control-flow-functions-and-recursion/
    README.md
    notes.md
    mistakes.md
    questions.md
    src/
    tests/
  05-arrays-strings-and-character-handling/
    README.md
    notes.md
    mistakes.md
    questions.md
    src/
    tests/
  06-pointers-const-and-pointer-arithmetic/
    README.md
    notes.md
    mistakes.md
    questions.md

```

```
src/
tests/
07-structs-unions-enums-and-typedefs/
  README.md
  notes.md
  mistakes.md
  questions.md
  src/
  tests/
08-dynamic-memory-lifetime-and-ownership/
  README.md
  notes.md
  mistakes.md
  questions.md
  src/
  tests/
09-preprocessor-headers-modular-design-and-builds/
  README.md
  notes.md
  mistakes.md
  questions.md
  src/
  tests/
10-file-handling-binary-data-and-parsing/
  README.md
  notes.md
  mistakes.md
  questions.md
  src/
  tests/
11-bit-manipulation-endianness-and-low-level-representation/
  README.md
  notes.md
  mistakes.md
  questions.md
  src/
  tests/
12-function-pointers-callbacks-and-generic-programming/
  README.md
  notes.md
  mistakes.md
  questions.md
  src/
  tests/
13-data-structures-in-c/
  README.md
  notes.md
  mistakes.md
  questions.md
  src/
  tests/
14-algorithms-complexity-and-problem-solving-in-c/
  README.md
  notes.md
  mistakes.md
  questions.md
  src/
  tests/
15-debugging-testing-sanitizers-and-tooling/
```

```
README.md
notes.md
mistakes.md
questions.md
src/
tests/
16-system-programming-concurrency-embedded-c-and-expert-level-pitfalls/
README.md
notes.md
mistakes.md
questions.md
src/
tests/
```

Brutal rule

If someone says you are weak at C, the answer is not emotion. The answer is reps, review, debugging, and visible proof. Finish this document and your GitHub itself will answer them.